

ISIT312/ISIT912 Big Data Management

Spring 2025

Spark

In this practice, you will perform basic operations and develop basic data processing applications in Spark.

Warning: To copy the fragments of code from this document to your working *Terminal* use *evince pdf viewer*.

Software Installation and Setup

If you use a system installed in 3.230 i.e. a laboratory room for ISIT312/912, then you do not need to install VirtualBox.

If you use your system (laptop) then start from installation of the latest version of VirtualBox and the extension pack (optional). The distribution packages are available at <https://www.virtualbox.org/wiki/Downloads> for Windows, MacOS (Intel processor only) and Linux.

If you use your system (laptop) then you must copy a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1.ova` located at `C:\Program Files (x86)\ISIT312` and `ISIT912` (if you cannot find the file in this location please ask a tutor about correct location). You can also download the file from Moodle.

Next, start VirtualBox and import the file. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

After the import is completed, a virtual machine named `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1` appears in the VirtualBox.

If you use a system installed in 3.320 then start VirtualBox and import a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1.ova` located at `C:\Program Files (x86)\ISIT312` and `ISIT912` folder into VirtualBox. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

After the import is completed, a virtual machine named `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1` appears in the VirtualBox.

Laboratory Instructions

Start a virtual machine `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1`

Note, that both the account name and the password are: `bigdata`

(0) Start Shell

After you log on the Ubuntu system with a user name `bigdata` and password `bigdata`, start a Terminal window with `Ctrl + Alt + T` or use the fifth from top icon located at a vertical stripe (sidebar) on the left-hand side of the screen.

The documents `LinuxCommandLineCheatSheet.pdf` and `Efficient-Linux-at-the-Command-Line-ch4.pdf` available through [Readings](#) link in [Resources](#) section in [linux](#) folder on Moodle contain more information on how to use Linux Shell in Terminal window.

You can use the Terminal window to interact with Hadoop. A simple hint that may make your life much easier is to use "Up" and "Down" keys on a keyboard to navigate through the commands already processed in Terminal window.

Copy and Paste in Terminal window can be done with the keys `Ctrl/Insert` and with the keys `Shift/Insert` or with a mouse rightclick.

Now you can interact with Hadoop.

(1) Hadoop Initialisation

Now you can start Hadoop. Process a shell script `start-all.sh` in Terminal window in the following way:

```
$HADOOP_HOME/sbin/start-all.sh
```

The system will display few warnings and the following messages:

```
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bigdata-VirtualBox]
Starting resourcemanager
Starting nodemanagers
```

After you ignore the warning messages you can view the running processes with a command:

```
jps
```

The following results should be returned (note that the process numbers may be different):

```
819  NodeManager
2501  SecondaryNameNode
2310  DataNode
4391  Jps
2187  NameNode
2700  ResourceManager
```

(2) Spark Initialisation

Now you can start Spark. Process a shell script `start-all.sh` in Terminal window in the following way:

```
$SPARK_HOME/sbin/start-all.sh
```

The system will display few warnings and the following messages:

```
starting org.apache.spark.deploy.master.Master, logging to
/home/bigdata/spark-3.5.1-bin-hadoop3/logs/spark-bigdata-
org.apache.spark.deploy.master.Master-1-bigdata-VirtualBox.out
localhost: starting org.apache.spark.deploy.worker.Worker, logging
to /home/bigdata/spark-3.5.1-bin-hadoop3/logs/spark-bigdata-
org.apache.spark.deploy.worker.Worker-1-bigdata-VirtualBox.out
```

After you ignore the warning messages you can view the running processes with a command:

jps

The following results should be returned (note that the process numbers may be different):

```
3634 SecondaryNameNode
3955 NodeManager
3447 DataNode
7207 Jps
4329 Master
4441 Worker
3310 NameNode
3839 ResourceManager
```

(3) How to start and interact with Spark in the Shark Shell?

To start the PySpark shell, process the following command in a Terminal window.

pyspark

PySpark is the Python Application Programming Interface for Apache Spark. PySpark displays the following messages.

```
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more
information.
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,
use setLogLevel(newLevel).
Welcome to
```



```
Using Python version 3.10.12 (main, Nov 20 2023 15:14:05)
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[*], app id = local-1716025752494).
SparkSession available as 'spark'.
```

(4) Creating and listing simple Resilient Distributed Datasets (RDD)s

To create a simple RDD, apply a method `parallelize()` to Spark context object `sc` in front of PySpark prompt `>>>`.

```
numbers = sc.parallelize([1, 2, 3, 4, 5])
```

To list the contents of RDD numbers apply a method `collect()` in front of `>>>` prompt.

```
numbers.collect()
```

Create another RDD and use print to lists its contents.

```
names = sc.parallelize(["James", "Harry", "Robin"])
print(names.take(3))
```

(5) Reading/writing RDDs to/from HDFS

Upload a file `NOTICE.txt` from `$HADOOP_HOME` to your home folder in HDFS. To do so perform the following actions in another Terminal window.

```
cd $HADOOP_HOME
hdfs dfs -put NOTICE.txt
```

Return to PySpark window. Read a file `NOTICE.txt` from HDFS to RDD in the following way.

```
notice_lines = sc.textFile("NOTICE.txt")
```

To verify the results, list the contents of RDD `notice_lines`.

```
notice_lines.collect()
```

Upload a file `README.txt` located in `$HADOOP_HOME` to a folder `/user` in HDFS.

```
hdfs dfs -put README.txt /user
```

List the contents of `README.txt` located in HDFS in `/user` folder.

```
hdfs dfs -cat /user/README.txt
```

Create RDD from a file located in HDFS. This time use a complete address of a file in HDFS.

```
read_lines = sc.textFile("hdfs://localhost:9000/user/README.txt")
```

Save RDD `read_line` in a folder `/user/readme` in HDFS (write it back to HDFS).

```
read_lines.saveAsTextFile("hdfs://localhost:9000/user/readme")
```

Check the contents of a folder `/user/readme` and the contents of a file saved in `/user/readme`.

```
hdfs dfs -ls /user/readme
hdfs dfs -cat /user/readme/p*
```

(6) Counting lines, filtering lines and counting words in RDD

List the first line in RDD `read_lines` in the following way.

```
read_lines.first()
```

Count the total number of lines in RDD `read_lines`.

```
read_lines.count()
```

List all lines that contain a word `apache` in RDD `read_lines`.

```
read_lines.filter(lambda x: "apache" in x).collect()
```

Use a text editor to create a file `words.txt` and upload the file to your home folder (`/user/bigdata`) in HDFS. For example, you can save the following lines in a file `words.txt`.

```
aaa aaa
bb bb bb
cc cc ccc
```

Next, create RDD with the lines included in a file `words.txt`.

```
lines_rdd = sc.textFile("words.txt")
```

Word counting task can be implemented by the following sequence operations on RDD `lines_rdd`. Process the lines listed below in PySpark window.

```
counters = (lines_rdd.flatMap(lambda line: line.split(" "))
             .map(lambda word: (word, 1))
             .reduceByKey(lambda a, b: a + b) )
```

And then, list the contents of RDD `counters`.

```
counters.collect()
```

(7) Implementation of self-contained applications

Open a new Terminal window and use a text editor to create a file `SimpleApp.py` with the following contents.

```
"""SimpleApp.py"""
from pyspark.sql import SparkSession

logFile = "README.txt"
spark = SparkSession.builder.appName("SimpleApp").getOrCreate()

logData = spark.read.text(logFile).cache()
numAs = logData.filter(logData.value.contains('a')).count()
numBs = logData.filter(logData.value.contains('b')).count()
print("Lines with a: %i, lines with b: %i" % (numAs, numBs))
spark.stop()
```

It is a self-contained application that processes a file `README.txt` located in your home folder on HDFS. The application counts the total number of lines in a file `README.txt` that contain a letter `a` and that contain a letter `b`.

Run the application in the same Terminal window in the following way.

```
$SPARK_HOME/bin/spark-submit --master local[4] SimpleApp.py
```

Wait a moment for the results.

List a file `README.txt` to verify the results.

```
hdfs dfs -cat /user/bigdata/README.txt
```

Another self-contained application finds the total length of all lines included in a file `README.txt`.

Open a text editor and create a file `AnotherApp.py` with the following contents.

```
"""AnotherApp.py"""
from pyspark import SparkConf, SparkContext

spark_conf = SparkConf()
sc = SparkContext(conf=spark_conf)

lines = sc.textFile("README.txt")
lineLengths = lines.map(lambda s: len(s))
totalLength = lineLengths.reduce(lambda a, b: a + b)
print(totalLength)

sc.stop()
```

Run the application in the same Terminal window in the following way.

```
$SPARK_HOME/bin/spark-submit --master local[4] AnotherApp.py
```

Wait a moment for the results.

List a file `README.txt` to verify the results.

```
hdfs dfs -cat /user/bigdata/README.txt
```

(8) Processing various operations on RDDs

Use a file `README.txt` located in your home folder on HDFS. Read the file into RDD and split it into words.

```
lines = sc.textFile("README.txt")
words = lines.flatMap(lambda line: line.split(' '))
```

Eliminate duplicates, count the words and filter the words

```
distinct_words = words.distinct()
distinct_words.count()
a_words = words.filter(lambda word: word.startswith("a"))
```

Sorting results

```
sorted(sc.parallelize([1,2,5,3,4]).collect())
```

Sort with `sortBy` in ascending and descending orders

```
sc.parallelize(['a','d','c','e','b']).sortBy(lambda x: ord(x)).collect()
sc.parallelize([1,2,5,3,4]).sortBy(lambda x: x).collect()

sc.parallelize(['a','d','c','e','b']).sortBy(lambda x: -ord(x)).collect()
sc.parallelize([1,2,5,3,4]).sortBy(lambda x: -x).collect()
```

Sorting pairs by the first and the second element of each pair

```
sc.parallelize([(1,3), (2,5), ('a',1), ('b',2), ('d',4)]).sortBy(lambda x: x[0]).collect()
sc.parallelize([(1,3), (2,5), ('a',1), ('b',2), ('d',4)]).sortBy(lambda x: x[1]).collect()
```

Sorting pairs with `sortByKey` in ascending and descending order

```

sc.parallelize([('ab',1),('aa',2),('1',3),('d',4),('2',5)]).sortByKey(True,1).collect()
sc.parallelize([('ab',1),('aa',2),('1',3),('d',4),('2',5)]).sortByKey(False,1).collect()

tmp2 = [('Mary',1),('had',2),('a',3),('little',4),('lamb',5)]
tmp2.extend([('whose', ),('fleece',7),('was',8),('white',9)])
sc.parallelize(tmp2).sortByKey(True, 1, keyfunc=lambda k: k.lower()).collect()
sc.parallelize(tmp2).sortByKey(False, 1, keyfunc=lambda k: k.lower()).collect()

```

reduce versus reduceByKey

```

from operator import add
sc.parallelize([1, 2, 3, 4, 5]).reduce(add)

sc.parallelize([("a",1), ("b",1), ("a", 1)]).reduceByKey(add).collect()

```

map versus flatMap

```

sc.parallelize(["a b c","c d e"]).map(lambda line: line.split(" ")).collect()
sc.parallelize(["a b c","c d e"]).flatMap(lambda line: line.split(" ")).collect()

```

Iteration with foreach

```

def f(x):
    print(x+1)

sc.parallelize([1,2,3,4,5]).foreach(f)

```

groupBy versus groupByKey

```

result = sc.parallelize([1,1,2,3,5,8]).groupBy(lambda x: x % 2).collect()
sorted([(x, sorted(y)) for (x, y) in result])

sorted(sc.parallelize([("a",1), ("b",1), ("a",1)]).groupByKey().mapValues(len).collect())

```

join operation

```

rdd1 = sc.parallelize([("a", 1), ("b", 4)])
rdd2 = sc.parallelize([("a", 2), ("a", 3)])
sorted(rdd1.join(rdd2).collect())

```

union operation

```

rdd = sc.parallelize([1, 1, 2, 3])
rdd.union(rdd).collect()

```

intersection operation

```

rdd1 = sc.parallelize([1, 10, 2, 3, 4, 5])
rdd2 = sc.parallelize([1, 6, 2, 3, 7, 8])
rdd1.intersection(rdd2).collect()

```

subtract operation

```

rdd1 = sc.parallelize([("a", 1), ("b", 4), ("b", 5), ("a", 3)])
rdd2 = sc.parallelize([("a", 3), ("c", None)])
sorted(rdd1.subtract(rdd2).collect())

```

(9) Creating and extending DataFrames

A simple DataFrame people can be created in the following way.

```

people = spark.createDataFrame([("James",25),("Harry",17),("Robin",40)],["name","age"])

```

The contents of a DataFrame `people` can be listed in the following way.

```
people.show()
```

A schema of a DataFrame `people` can be listed in the following way.

```
people.printSchema()
```

A new column can be added to a DataFrame in the following way.

```
from pyspark.sql.functions import col, when
new_people = people.withColumn("life_stage", when(col("age") < 13, "child")
                                     .when(col("age").between(13, 19), "teenager")
                                     .otherwise("adult") )

new_people.show()
new_people.printSchema()
```

(10) Reading/writing DataFrames to/from HDFS

Create a file `people.json` with the following contents.

```
{"name": "James", "age": 25}
{"name": "Harry", "age": 24}
{"name": "Robin", "age": 50}
```

Upload the file to your home folder on HDFS

```
hdfs dfs -put people.json
```

Create a DataFrame `jdf` in the following way.

```
jdf = spark.read.load("people.json", format="json")
```

Display a DataFrame `jdf`.

```
jdf.show()
```

Display a schema of DataFrame `jdf`.

```
jdf.printSchema()
```

Create a file `people.csv` with the following contents.

```
James, 25
Harry, 24
Robin, 50
```

and upload the file to your home folder on HDFS.

```
hdfs dfs -put people.csv
```

Create a DataFrame `cdf` in the following way.

```
cdf = spark.read.load("people.csv", format="csv")
```

Display a DataFrame `cdf`.

```
cdf.show()
```

Display a schema of DataFrame `cdf`.

```
cdf.printSchema()
```

Note that the names of columns `_c0` and `_c1` in DataFrame `cdf` were automatically created by a loader. Such names provide no semantics for the values included in the columns. We create a new DataFrame `sdf` with a more meaningful schema.

```
from pyspark.sql.types import *
CSVschema = StructType([StructField("name", StringType()),
                             StructField("age", IntegerType())])
sdf=spark.read.format("csv").schema(CSVschema)
                             .load("people.csv")

sdf.printSchema()
sdf.show()
```

Create a file `people.txt` with the following contents.

```
James 25
Harry 24
Robin 50
```

Upload the file to your home folder on HDFS.

```
hdfs dfs -put people.txt
```

Create a DataFrame `tdf` in the following way.

```
tdf = spark.read.text("people.txt")
```

Display a DataFrame `tdf` and its schema.

```
tdf.show()
tdf.printSchema()
```

Finally, we save the DataFrames `cdf` and `jdf` in HDFS and list the contents of the new files in HDFS.

```
cdf.write.save("cpeople", format="csv")
hdfs dfs -cat cpeople/p*

jdf.write.save("jpeople", format="json")
hdfs dfs -cat jpeople/p*
```

(11) Converting RDDs to DataFrames

Create an RDD `rdd` in the following way.

```
rdd = sc.parallelize([('James',25), ('Harry',24), ('Robin',50)])
```

Convert an RDD `rdd` into a `DataFrame` `df` in the following way.

```
df = spark.createDataFrame(rdd)
```

Display a `DataFrame` `df` and its schema.

```
df.show()
df.printSchema()
```

(12) Processing various operations on DataFrames

Create the `DataFrames` `people` and `department` in the following way.

```
people = spark.createDataFrame([
{"deptId": 1, "age": 40, "name": "Hyukjin Kwon", "gender": "M", "salary": 50},
{"deptId": 1, "age": 50, "name": "Takuya Ueshin", "gender": "M", "salary": 100},
{"deptId": 2, "age": 60, "name": "Xinrong Meng", "gender": "F", "salary": 150},
{"deptId": 3, "age": 20, "name": "Haejoon Lee", "gender": "M", "salary": 200} ])

department = spark.createDataFrame([
{"id": 1, "name": "PySpark"},
{"id": 2, "name": "ML"},
{"id": 3, "name": "Spark SQL"} ])
```

Process the following operations on `DataFrames` `people` and `department`. Use a method `show()` to list the contents of the new `DataFrames` created.

```
df1 = people.filter(people.age > 50)

df2 = people.where(people.age == 60)

df3 = people.filter('age > 30')

df4 = people.where('age = 60')

df5 = people.select('*')

df6 = people.select(people.name, (people.age + 10).alias('age'))

df7 = people.groupBy().avg()

df8 = people.groupBy("age").count()

df9 = people.groupBy("name").agg({"age": "sum"}).sort("name")

df10 = people.groupBy(people.name).max().sort("name")

df11 = people.groupBy(["name", people.age]).count().sort("name", "age")
```

Create the new `DataFrames` and process the following binary operations on the new `DataFrames`. List the contents of the new `DataFrames`. The binary operations include set union (`union`), intersection (`intersect`) and difference (`subtract`).

```

df12 = spark.createDataFrame([(1, 'A'), (2, 'B')], ['id', 'value'])
df13 = spark.createDataFrame([(3, 'C'), (4, 'D')], ['id', 'value'])
df14 = df12.union(df13)

df15 = spark.createDataFrame([("a", 1), ("a", 1), ("b", 3), ("c", 4)], ["C1", "C2"])
df16 = spark.createDataFrame([("a", 1), ("a", 1), ("b", 3)], ["C1", "C2"])
df17 = df16.intersect(df15).sort(df16.C1.desc())

df18 = spark.createDataFrame([("a", 1), ("a", 1), ("b", 3), ("c", 4)], ["C1", "C2"])
df19 = spark.createDataFrame([("a", 1), ("a", 1), ("b", 3)], ["C1", "C2"])
df20 = df18.subtract(df19)

```

Create the new DataFrames and process the following operations of join (`join`) and outer join (`join outer`) on the new DataFrames. List the contents of the new DataFrames.

```

from pyspark.sql import Row
from pyspark.sql.functions import desc

df1 = spark.createDataFrame([(2, "Alice"), (5, "Bob")]).toDF("age", "name")
df2 = spark.createDataFrame([Row(height=80, name="Tom"), Row(height=85, name="Bob")])
df3 = spark.createDataFrame([Row(age=2, name="Alice"), Row(age=5, name="Bob")])
df4 = spark.createDataFrame([Row(age=10, height=80, name="Alice"),
                             Row(age=5, height=None, name="Bob"),
                             Row(age=None, height=None, name="Tom"),
                             Row(age=None, height=None, name=None) ])

df21 = df1.join(df2, 'name').select(df2.name, df2.height)
df22 = df1.join(df4, ['name', 'age']).select(df4.name, df4.age)
df23 = df1.join(df2, df1.name == df2.name, 'outer')
        .select(df2.name, df2.height).sort(desc("name"))
df24 = df1.join(df2, 'name', 'outer').select('name', 'height').sort(desc("name"))

```

(13) Processing DataFrames with SQL

It is possible to process DataFrames with pure SQL. First, register a DataFrame as SQL temporary view in the following way.

```
people.createOrReplaceTempView("vpeople")
```

Next, we use `sql` method to process SQL statements on the temporary view created earlier.

```
sqlDF = spark.sql("SELECT * FROM vpeople")
sqlDF.show()
```

References

Spark Core and RDDs

<https://spark.apache.org/docs/latest/api/python/reference/pyspark.html>

Spark SQL and DataFrames

<https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html>